Custom Cell Design Engine

Ayush Garg & Rohan Prasad
June 6, 2026

CONTENTS

Executive summary

The Custom Cell Design Engine takes a battery requirement in plain language and produces a ranked set of buildable cell designs, plus a map of the design space, in a few minutes on an ordinary machine. It is built to tailor one of IBC's platform families fast and well, not to search every possible cell, and every number it reports is labeled with how much to trust it.

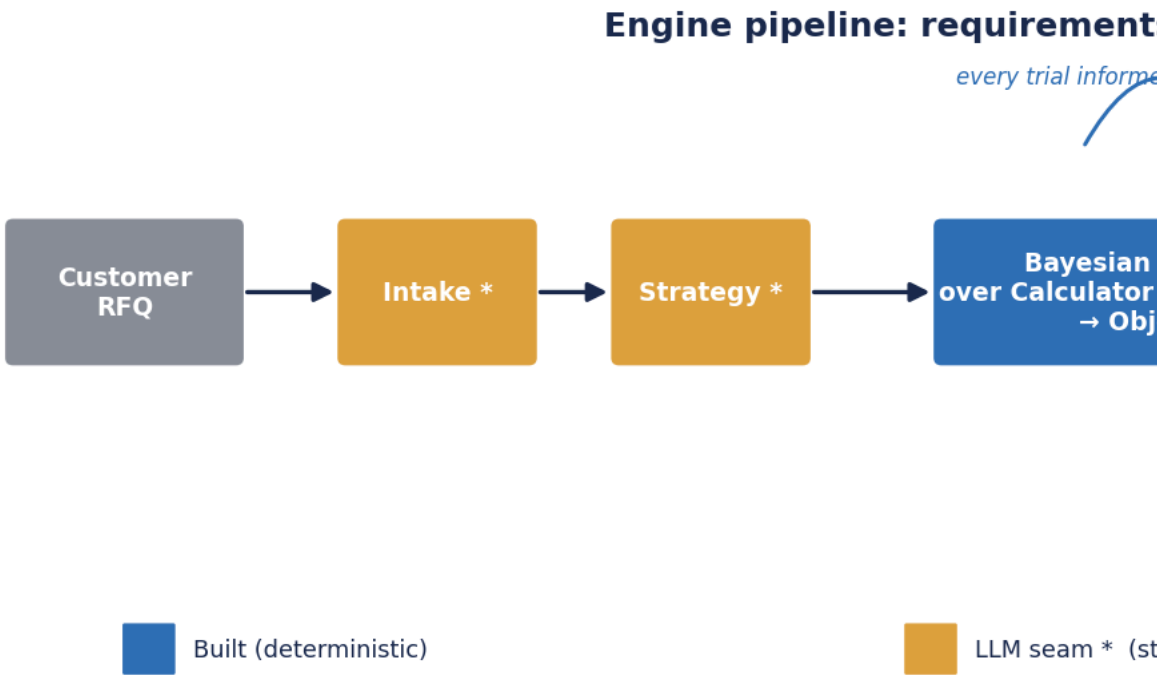
As of today the deterministic engine is built and runs end to end. The physics simulation has been validated against real published cell data. The result is demoable now on literature data, and it becomes deployable once IBC's own values are plugged in and the model is calibrated to IBC's cells. The pieces that remain are either owned by us as a next step or gated on IBC (real values, API funding, and test data for calibration).

The core principle throughout: the tool does not compete with the lab, it competes with the engineer's first guess. It makes the starting design better and makes each physical build teach more, so the existing build-test loop converges in fewer passes.

This report describes what has been built, what the numbers mean, how the work was hardened, and exactly what is done, not done, and next.

What the engine does

A requirement enters as plain application language. The engine turns it into a structured specification, searches the manufacturable design space with Bayesian optimization over a validated physics model, and returns ranked designs with the reasoning attached.



The pipeline is a deterministic Python program. Language-model agents are consulted only at defined checkpoints (intake, strategy, analysis, report), and they pick from bounded menus that a deterministic layer validates before anything runs. The physics is the final filter. This containment is deliberate: the worst a hallucination can cause is a rejected message and a retry, never a bad design reaching the output.

The amber boxes are those language-model checkpoints. Today they run as deterministic stubs with fixed input and output contracts, so the whole engine runs with no API key and no cost. Swapping in real language models later is a one-file change per checkpoint, because the contract the rest of the system depends on does not change.

The blue box is the heart: a Bayesian optimizer that maintains a statistical map of the design space and chooses the next design to simulate by balancing exploration against exploitation. Every trial is informed by every previous trial, so it finds good designs in tens of simulations rather than a blind grid of thousands.

What has been built

Five layers, bottom up, each tested before the next was started.

Quote-grade calculator (tier-1, trustworthy). Capacity, energy, mass, volume, and the anode-to-cathode balance from algebra over known material properties. This is bookkeeping, not prediction, so it lands within a percent or two of a built cell. These are the numbers that can go in front of a customer.

DFN physics simulation (validated). The Doyle-Fuller-Newman model, the industry-standard physics of a lithium-ion cell, run through PyBaMM. It reports rate capability and relative thermal behavior. The numerical solution is converged, and the model has been validated against real cell data (next section).

Objective and Bayesian optimizer. The objective maximizes the trustworthy specific-energy number subject to the application's power and thermal constraints. The optimizer searches the manufacturable space and finds the best feasible design in tens of simulations.

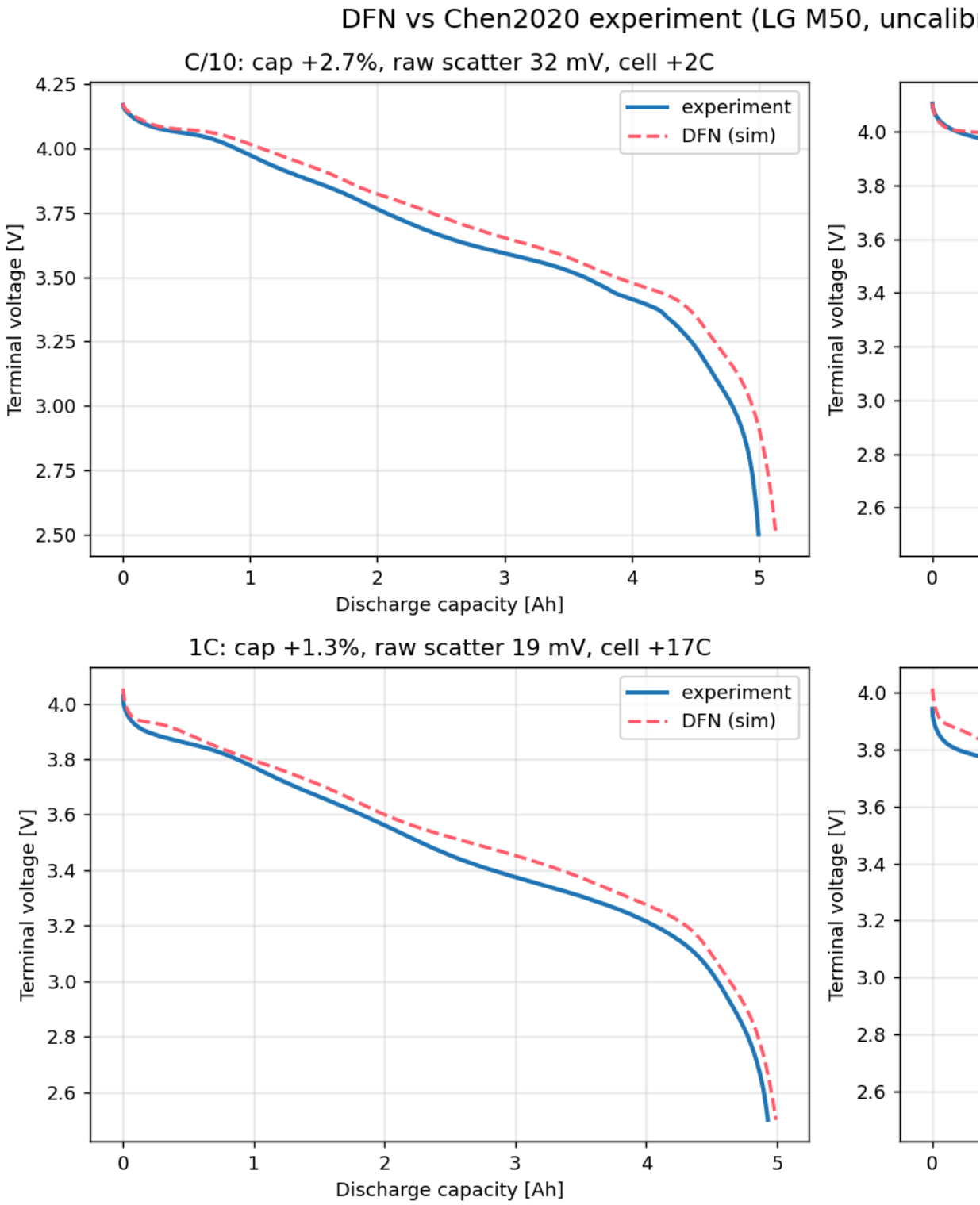
Orchestrator and report. A deterministic state machine runs the pipeline and emits ranked, distinct designs plus design-space heatmaps that give an engineer intuition about where good designs live and where constraints bind.

Agent seams. The four language-model checkpoints are named functions with fixed contracts, validated by the orchestrator, currently filled with deterministic logic. This is what makes the future language-model integration cheap and safe.

The whole system is configuration-driven. Plugging in IBC's real platforms, manufacturing limits, and example requests is editing a configuration file, not rewriting code.

Validation

The DFN model was validated against real LG M50 discharge data (the Chen2020 public dataset) at four discharge rates, on literature parameters.

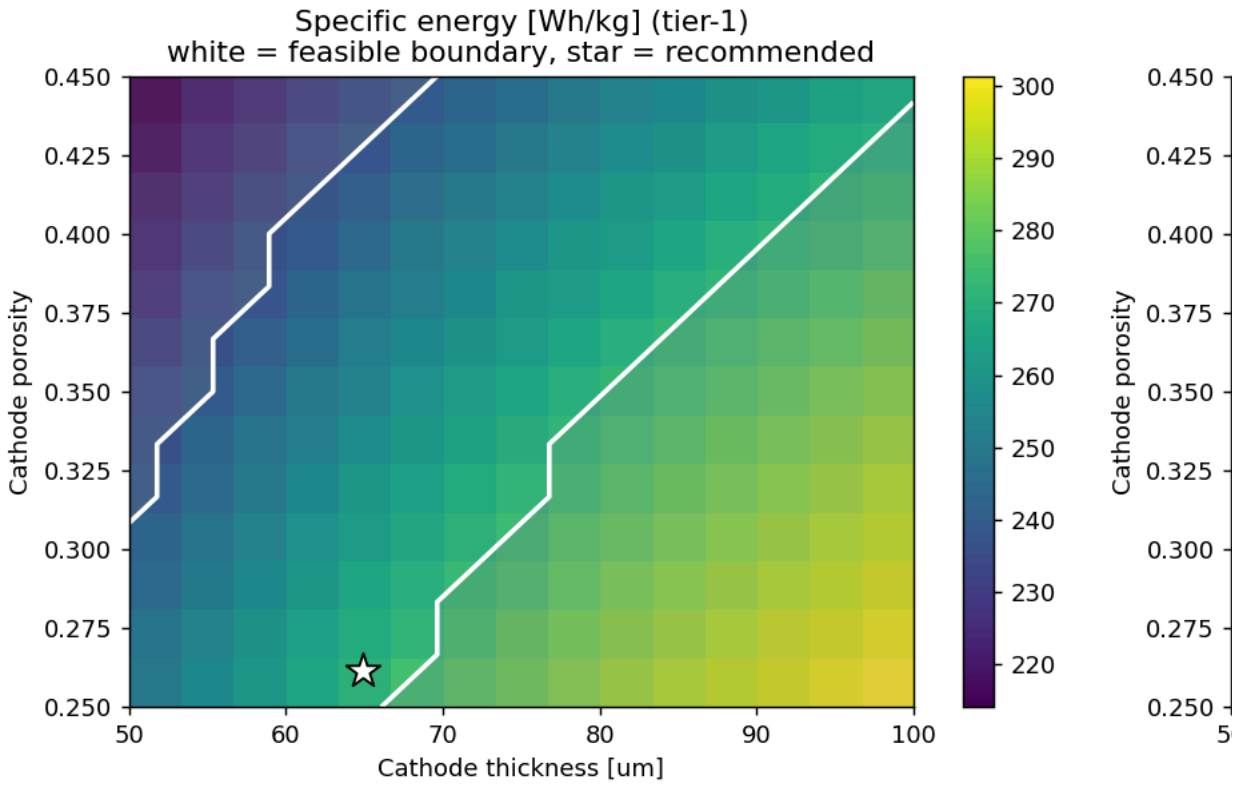


The honest reading. The physics shape is right: on the raw comparison the model tracks the real cell to roughly 30 mV at low rate and tighter at high rate. The dominant error is a near-constant offset of about 40 mV, which is exactly the kind of error that calibration removes. Two caveats we state openly: at the higher rates the comparison is partly temperature-confounded, because the real cell self-heats while our run is isothermal, and the flattering “tight” number depends on how the curves are normalized, so we report both versions.

What this establishes is that the machinery is sound and the physics shape is reasonable. It is not a calibrated, lab-grade claim about IBC’s cells. Calibration on IBC’s own test data is the step that converts these directional numbers into design-guidance numbers, and that step is the part an outside vendor cannot replicate, because the data never leaves IBC.

The demo result

Run end to end on an example drone requirement (peak discharge at 3C, minimum specific energy, power and thermal limits), the engine produces this.



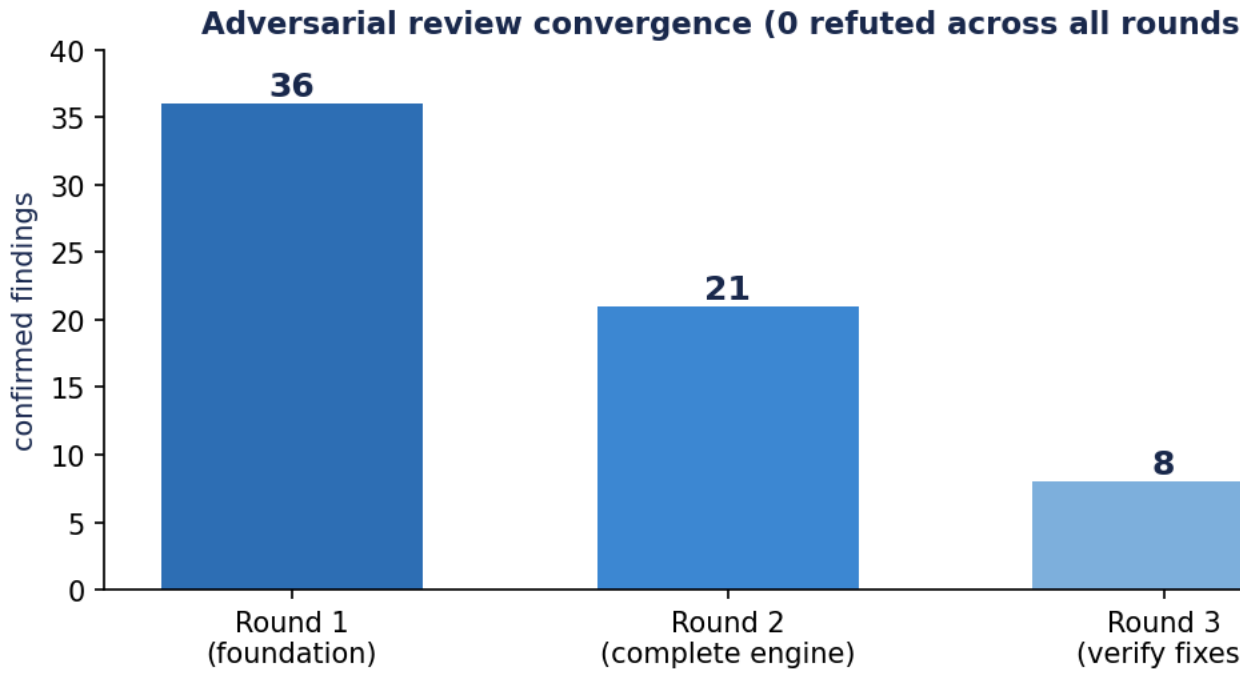
Recommended design is constraint-binding: rate 0.59 (floor 0.50), temp rise 41 C (limit 45, di

The left panel is the specific-energy landscape; the right panel is the optimizer score with its sampled path. The story an engineer reads in one glance: energy rises toward thick, dense electrodes (bottom right), but that corner is infeasible because it cannot deliver the power or stay within the thermal limit. The best feasible design therefore sits on the constraint boundary, and the optimizer finds it. The brightest energy cells are deliberately outside the feasible region.

The recommended design comes out around 270 Wh/kg and 24.5 Ah, meeting all targets, alongside a short list of distinct alternatives that trade a little energy for power and thermal margin. Every number carries its trust tier: the energy and capacity are quote-grade, the rate and thermal are directional.

How it was hardened

Before building on the foundation, and again after completing the engine, the whole system was put through adversarial review: independent agents looked for physics errors, misleading metrics, and overclaims, and every finding was then verified by a separate skeptic that tried to refute it.



The first round found 36 confirmed issues, including deep physics bugs that would not have survived a battery engineer's scrutiny: the calculator and the simulation were modeling two different cells, a porosity setting produced physically impossible electrodes, and material densities were off by about 30 percent. The second round, on the completed engine, found 21, led by one real correctness bug in how feasibility was ranked. The third round found 8, almost all presentation polish. Across all three rounds, zero findings were refuted as false. The trend, 36 to 21 to 8, is genuine convergence on a sound system. There are 47 automated tests, including the property test that would have caught the ranking bug.

Done, not done, and next

Done. The deterministic engine runs end to end. The calculator is quote-grade. The DFN is validated and numerically converged. The optimizer finds the feasible optimum. The report produces ranked designs and design-space maps. The agent seams are in place as validated stubs. 47 tests pass.

Not done (by design, for this stage). The engine runs on literature placeholder values, not IBC's. The language-model agents are deterministic stubs, not live models. The simulation is validated but not yet calibrated to IBC cells, so everything above the calculator is directional. There is no pack-level thermal model yet, so the absolute temperature is a relative proxy.

Next, in order.

1. ASSUMPTIONS.md sign-off and the strawman cell numbers (ours, this week).
2. The Kunal conversation to size the business case and confirm the process assumptions (the highest-leverage open item).
3. Plug in IBC's real platform values, manufacturing envelope, and example requirements (configuration, once we have them).
4. Fund and wire the live language-model agents (one file per seam).
5. Calibrate the DFN to IBC test data with PyBOP, the step that turns directional into design-guidance. This is the moat.
6. Add the pack thermal model and re-run the high-rate validation.
7. The DoE build plan: propose the few prototypes that jointly teach the most.

Honest limitations

We are explicit about what the tool does not do, because that honesty is what makes an engineer trust the rest.

It does not predict cells perfectly; nothing short of building and testing them does, which is why the lab remains the final word. It is uncalibrated today, so its absolute non-calculator numbers are wrong for IBC's specific cells. The DFN error grows at high discharge rates and near the end of discharge at high temperature, a known weak zone in the literature as well. Cycle life is always treated as a ranking with reasoning, never a promised number, because that is the figure physics defends least. And it does not fix yield, which is likely IBC's real production bottleneck; it helps the front of the business and frees pilot-line slots, and we do not claim more than that.

What we need from IBC

Three numbers size the entire value case, and all three live only inside IBC: how many custom requests come in per year, how many build-test loops a typical engagement takes and what one loop costs, and what the current request-to-design process actually looks like. The conversation with Kunal answers these. Beyond that, calibration needs IBC test data at sufficient richness, and the language-model layer needs the API funded as a small project cost. With those, the tool moves from a working demonstration to a calibrated instrument embedded in IBC's workflow.